As the unavoidable language of the web, as the language used to create experiences in the web browser from its very inception, JavaScript is a melting pot of different language ideas. It's not a language used exclusively by JavaScript programmers; it's a language frequently used by people who typically program in another language. JavaScript is a language used by people who don't necessarily understand (or want to understand) every subtle nuance. Because of all these different people who use JavaScript, it has necessarily changed substantially over time. At least, the way it is used, the techniques, and the types of things written in JavaScript have changed substantially. Not the syntax, though; that hasn't changed much at all.

JavaScript was created in about the time it takes to get over a cold. The short time frame in which JavaScript was created (and subsequently standardized) led to some problems being set into the language—problems that are still in the process of being fixed many years later. So, take a language that was created in a matter of days, add in some competing ideas, give it to a whole lot of people who don't know it very well, and what happens? You can fill in the rest of *that* story yourself.

How does CoffeeScript fit into this picture? CoffeeScript changes JavaScript's syntax. CoffeeScript simplifies JavaScript so that it expresses those key ideas borrowed from Self and Scheme, as understood by users of the language today. In doing so, it also fixes some of the problems caused by JavaScript's quick birth. Sure, as with all things, these changes come with their own unique set of trade-offs. To see if those trade-offs are right for you, it's time to see what CoffeeScript does to JavaScript syntax and how it arrived at the changes that it did. Doing this starts with a simple thought experiment. What syntax can be taken away from a JavaScript program while still leaving the meaning intact?

1.5. Creating CoffeeScript

CoffeeScript starts with an experiment to remove as much nonessential syntax from JavaScript as possible. By working through that experiment, you can begin to understand CoffeeScript syntax and see how it differs from